

Prompt 1 - Define User Stories

- **Purpose of AI Use:** Brainstorming and requirements to start project
- **AI-Suggested Content:**
- **Validation:** Read through and found some User Stories are too broad in the feature, and more detail is needed to be added and priorities to be moved around.
- **Decision:** Partially Accepted
- **Reflection:** AI helped give a starting point for the project, and to start spreading the work with the team
- **Responsible Person:** Paul Fourment

Prompt

Here is a project given, I will be working with a team, repo has been setup and a basic svelte kit app has been created, no database setup yet or anything else started. Per the given instructions give 10 user stories that we can use to start.

(User Story instructions and Project Description Pasted From PDF)

Response

Here are 10 user stories covering the core CareerConnect features. The first four are ordered so your Sprint 1 demo (Registration/Login + Profile/Resume) is covered. Each follows the Appendix B format, with acceptance criteria and a starting task breakdown you can turn into

Summary

ID	Title	Primary User	Priority	Sprint
US-01	Account registration	Job Seeker / Recruiter	High	1
US-02	Login and logout	Job Seeker / Recruiter	High	1
US-03	Profile management	Job Seeker	High	1
US-04	Resume upload and management	Job Seeker	High	1
US-05	Job posting management	Recruiter	High	TBD
US-06	Job search and filtering	Job Seeker	High	TBD
US-07	Job application submission	Job Seeker	High	TBD
US-08	Application status tracking	Recruiter	Medium	TBD
US-09	Application history dashboard	Job Seeker	Medium	TBD
US-10	Saved jobs	Job Seeker	Medium	TBD

US-01: Account registration

Priority: High · **Sprint:** 1

As a job seeker, I want to create an account with my email and a password so that I can access the platform's features.

Acceptance criteria

- ☐ Email must be unique and valid.
- ☐ Password meets minimum strength rules.
- ☐ The user chooses a role (Job Seeker or Recruiter).
- ☐ A clear error message is shown for invalid input.
- ☐ After signup, the user is logged in or redirected to the login page.

Tasks

- **Task 1.1:** Choose the database/auth approach and design the `users` schema (id, email, password hash, role, created_at).
 - **Task 1.2:** Implement the signup endpoint/form action with server-side validation and password hashing.
 - **Task 1.3:** Build the registration page UI in SvelteKit with client-side validation.
 - **Task 1.4:** Write tests for valid signup, duplicate email, and weak password.
-

US-02: Login and logout

Priority: High · **Sprint:** 1

As a registered user, I want to log in and log out securely so that my personal data stays protected.

Acceptance criteria

- ☐ Correct credentials create a session.
- ☐ Wrong credentials show a generic error message.
- ☐ Protected routes redirect unauthenticated users to the login page.
- ☐ Logout ends the session.

Tasks

- **Task 2.1:** Implement session handling (e.g. in

`hooks.server.ts`).

- **Task 2.2:** Build the login page and logout action.
 - **Task 2.3:** Add route guards for authenticated pages and role-based pages.
 - **Task 2.4:** Write tests for login success, login failure, and protected-route redirects.
-

US-03: Profile management

Priority: High · **Sprint:** 1

As a job seeker, I want to create and edit my profile (name, location, skills, education, experience summary) so that recruiters and the platform can understand my background.

Acceptance criteria

- ☐ The user can view and edit profile fields.
- ☐ Changes persist after a page refresh.
- ☐ Required fields are validated.
- ☐ Users can only edit their own profile.

Tasks

- **Task 3.1:** Design the `profiles` schema linked to `users`.
 - **Task 3.2:** Implement load and update logic for the profile page.
 - **Task 3.3:** Build the profile view/edit UI.
 - **Task 3.4:** Test authorization so one user cannot modify another user's profile.
-

US-04: Resume upload and management

Priority: High · **Sprint:** 1

As a job seeker, I want to upload, view, replace, and delete my resume so that I can use it when applying to jobs.

Acceptance criteria

- ☐ Only PDF (and optionally DOCX) files under a size limit are accepted.
- ☐ The uploaded resume can be downloaded or previewed.
- ☐ The user can replace or delete the resume.
- ☐ Files are only accessible to the owner (and to recruiters for jobs the user applied to).

Tasks

- **Task 4.1:** Choose file storage (cloud bucket, or local storage for development).
 - **Task 4.2:** Design the `resumes` schema (file path, file name, upload date, owner).
 - **Task 4.3:** Implement upload/delete endpoints with file type and size validation.
 - **Task 4.4:** Build the resume management UI.
 - **Task 4.5:** Test invalid file types, oversized files, and access control.
-

US-05: Job posting management

Priority: High

As a recruiter, I want to create, edit, and close job postings so that job seekers can find and apply to my openings.

Acceptance criteria

- ☐ Postings include title, company, location, job type, description, requirements, and application deadline.
- ☐ Only recruiters can create postings.
- ☐ Recruiters can only edit their own postings.
- ☐ Closed postings no longer accept applications.

Tasks

- **Task 5.1:** Design the `jobs` schema.
 - **Task 5.2:** Implement create/update/close actions with role checks.
 - **Task 5.3:** Build the recruiter posting form and "My Postings" page.
-

US-06: Job search and filtering

Priority: High

As a job seeker, I want to search jobs by keyword and filter by location, job type, and posting date so that I can quickly find relevant opportunities.

Acceptance criteria

- ☐ Keyword search matches job title, company, and description.
- ☐ Filters can be combined.
- ☐ Results show key details and link to a job detail page.
- ☐ An empty state is shown when there are no results.

Tasks

- **Task 6.1:** Implement the search xquery with filters and pagination.
 - **Task 6.2:** Build the search page with filter controls and URL query parameters.
 - **Task 6.3:** Build the job detail page.
-

US-07: Job application submission

Priority: High

As a job seeker, I want to apply to a job using one of my uploaded resumes and an optional cover note so that I can submit applications

directly through the platform.

Acceptance criteria

- ☐ The user selects a resume before applying.
- ☐ Duplicate applications to the same job are prevented.
- ☐ The application is recorded with status "Applied".
- ☐ The recruiter can see the application.

Tasks

- **Task 7.1:** Design the `applications` schema (job, applicant, resume, status, timestamps).
 - **Task 7.2:** Implement the apply action with duplicate prevention.
 - **Task 7.3:** Build the apply flow on the job detail page.
 - **Task 7.4:** Build the recruiter view of applicants for a posting.
-

US-08: Application status tracking

Priority: Medium

As a recruiter, I want to update an application's status (Applied, Interview, Offered, Rejected) so that applicants know where they stand.

Acceptance criteria

- ☐ Only the posting's recruiter can change the status.
- ☐ Status changes are timestamped.
- ☐ The applicant sees the updated status.

Tasks

- **Task 8.1:** Implement the status update action with authorization.
 - **Task 8.2:** Store status history (status + changed_at).
 - **Task 8.3:** Add status controls to the recruiter applicant view.
-
-

US-09: Application history dashboard

Priority: Medium

As a job seeker, I want a dashboard listing all my applications with their current status so that I can stay organized.

Acceptance criteria

- ☐ Applications show job title, company, date applied, and current status.
- ☐ The list can be filtered by status.
- ☐ Summary counts per status are shown.

Tasks

- **Task 9.1:** Implement a dashboard data loader.
 - **Task 9.2:** Build the dashboard UI with status filters and summary counts.
-

US-10: Saved jobs

Priority: Medium

As a job seeker, I want to save jobs to a favourites list so that I can review and apply to them later.

Acceptance criteria

- ☐ The user can save and unsave jobs from search results and the job detail page.
- ☐ Saved jobs appear on a dedicated page.
- ☐ Closed postings are clearly marked.

Tasks

- **Task 10.1:** Design the `saved_jobs` schema.

- **Task 10.2:** Implement save/unsave actions.
 - **Task 10.3:** Build the Saved Jobs page and save toggle component.
-

Not Covered (left for team brainstorming)

- Deadline notifications and reminders
 - The required Generative AI feature
 - The team's original feature
-
-

Profile Management (US-04)

- **Purpose of AI Use:** Code generation for profile management page and the main page layout
 - **AI-Suggested Content:** App layout with an account sidebar, routes for recruiters and job seekers, profile/experience/education tables (SQL to run on Supabase), profile view and section edit dialogs, photo upload to Supabase Storage, change email/password, delete account, recruiter view of a profile, page transitions, school/company/city/skill autocomplete with a custom dropdown
 - **Validation:** Tested the profile management page, editing details and found it very clunky, added more prompts to make it a smoother experience and add all the sections needed for the future.
 - **Decision:** Accepted / Modified
 - **Reflection:** AI helped create the page and DB layout for the founding details for Job Seekers and Recruiters, with things like experience, education, skills, etc.
 - **Responsible Person:** Paul Fourment
-

Prompt 1

I have created a new branch for feature/profile-management. Here is the user story to follow. I know the main page hasn't been sorted out yet, but if we can do a layout for it that would be great. It doesn't need content yet, just a simple layout: account card on the side, the jobs and stuff will be in the center.

(US-04 user story pasted: view and edit name, profile picture, email, password, phone number, location, skills, education and experience summary; changes persist; required fields show clear validation errors; users can edit only their own profile. Tasks 4.1–4.4.)

Response 1

Recruiters vs job seekers: use the same routes and let each page adapt to `user.role` (e.g. the home page shows "Jobs for you" or "Your job postings", the profile shows skills or company). Separate routes only make sense for pages one role must never see (e.g. a future recruiter-only "post a job" page), which get a one-line role check. Two parallel route trees would duplicate the layout, profile page and guards.

Layout (`src/routes/(app)/+layout.svelte`): every logged-in page shares a glass account card on the left (photo or initials, name, headline or job title, location, Home/Profile links, Log out) with the page content in the center. On phones the card moves to the top. Logout became its own `/logout` route so it works from any page.

Data (Task 4.1): a `profile` table with one row per user, linked to `user`. The onboarding answers moved from `user` into it, so `user` keeps only account data.

Profile page (Tasks 4.2–4.3): view mode with an Edit button; edit form for photo, name, phone, location (city autocomplete), headline/skills/education/experience for job seekers, company/job title for recruiters;

separate forms to change email and password.

- Photo upload goes to a Supabase Storage bucket through its REST API from the server (secret key never reaches the browser), limited to PNG/JPG/WebP under 2 MB.
- Every action uses the logged-in user's id, never an id from the form, so users can only edit their own profile.

Tests (Task 4.4): validation moved into a pure function with unit tests using Node's built-in test runner (`pnpm test`), no new dependency.

Prompt 2

Can you add a simple page transition animation, quick and simple?

Response 2

Used the browser's View Transitions API through SvelteKit's `onNavigate` (the pattern from the SvelteKit docs, no library):

- The old page fades out (120 ms) and the new one fades in with a 6 px lift (200 ms).
- The account card is excluded from the transition so it stays still.
- Disabled for users with reduced motion enabled; browsers without support (Firefox) navigate normally.

Prompt 3

I want to avoid coming back to this page as much as possible, so I want

to finalize it as much as possible right now. Remember the project description we were given: what other things would we need in the profile management area? For things like education, companies and skills, I'd like to make them simpler.

- Education: use possible APIs to autofill and show known schools
- Companies: same thing (like LinkedIn has) if an API is available
- Skills: how LinkedIn organizes skills

(Project description pasted)

Response 3

Data sources (tested live)

Field	Source	Result
Schools	Hipo university list (free, no key)	Good coverage incl. Québec CEGEPs; HTTP-only, so our server fetches it (/api/schools)
Companies	Clearbit autocomplete (free, no key)	Names + websites; logos shown from the company website's favicon
Skills	ESCO (EU skills taxonomy)	Rejected: "react" returned "react calmly in stressful situations"

Skills like LinkedIn: standard skill names picked from a search box, shown as a list, with up to 5 starred top skills shown first. Implemented with a built-in list of about 415 common skills (technical, tools, business, soft skills, languages), with custom skills still allowed.

Additions proposed for the profile (chosen: all except resume, which stays a placeholder for its own story):

1. Structured experience entries (title, company, employment type,

- location, start/end month or current, description)
2. Structured education entries (school, degree, field of study, years, description)
 3. Job preferences (open to work, desired roles, job types, work mode, preferred locations), for job search filtering and matching later
 4. Read-only recruiter view `/profile/[id]`, since the story says recruiters should understand the candidate's background
 5. About section, links (LinkedIn, GitHub, portfolio), profile completeness bar, delete account

Implementation

- Profile organised like LinkedIn: each section is a card with its own Edit/Add button that opens a dialog. The open dialog is in the URL (e.g. `/profile?experience=new`), so back/Esc close it.
- Validation rewritten as one parser per section (dates in order and not in the future, years in range, links normalised, unknown choices dropped), covered by 6 unit test groups.
- Recruiter view: only recruiters can open it and only for job seekers; everyone else gets the same "not found" page so ids can't be probed.
- Delete account requires typing DELETE and the password; profile, experience and education rows are removed by the database's cascading deletes, and the photo is deleted from storage.

SQL to run in the Supabase SQL Editor

```
alter table public.profile
  drop column education,
  drop column experience,
  add column about                text,
  add column linkedin_url         text,
  add column github_url           text,
  add column website_url          text,
  add column company_domain       text,
  add column top_skills            text[] not null default
  add column open_to_work         boolean not null default
  add column desired_roles        text[] not null default
  add column job_types            text[] not null default
```

```

add column work_modes          text[]  not null default
add column preferred_locations text[]  not null default

create table public.experience (
  id          uuid primary key default gen_random_uuid()
  user_id     text not null references public."user"(id)
  title       text not null,
  company     text not null,
  company_domain text,
  employment_type text,
  location    text,
  start_date  date not null,
  end_date    date,
  description  text,
  created_at  timestamp not null default now(),
  constraint experience_dates_check check (end_date is null or end_date > start_date)
);
create index experience_user_id_idx on public.experience(user_id);
alter table public.experience enable row level security;

create table public.education (
  id          uuid primary key default gen_random_uuid()
  user_id     text not null references public."user"(id)
  school      text not null,
  school_domain text,
  degree      text,
  field_of_study text,
  start_year  integer,
  end_year    integer,
  description  text,
  created_at  timestamp not null default now(),
  constraint education_years_check check (end_year is null or end_year > start_year)
);
create index education_user_id_idx on public.education(user_id);
alter table public.education enable row level security;

```

Prompt 4

The preview of the suggestions is like the default browser dropdown and doesn't show the logos. Would it be possible to do it like LinkedIn, where it shows that information, so the options match the aesthetic?

Response 4

Replaced the browser's `<datalist>` with a custom Combobox component following the accessible combobox pattern (WAI-ARIA):

- Glass-styled list showing a logo, name and detail line: companies with their website, schools with province/country, cities with region/country; skills filter instantly with prefix matches first.
- Keyboard: arrow keys move, Enter picks, Esc closes the list without closing the dialog; in chip fields Enter on custom text still adds it.
- The list is rendered as a popover (top layer), so it appears above the dialog without being clipped, and flips above the field when there isn't room below.
- "Remote" is only offered when it matches what's typed.

Testing: checked in the browser: "shop" → Shopify / shopify.com with logo, keyboard selection stores the company's domain, "sherb" → Sherbrooke / Quebec, Canada, "py" → Python, pytest, PyTorch, and Esc behaviour inside the dialog.

First-Login Onboarding (US-16)

- **Purpose of AI Use:** Brainstorming a small second feature for Sprint 1, then code generation for it
- **AI-Suggested Content:** The onboarding modal idea and its user story, a role-specific onboarding modal, save/skip form actions,

onboarding columns on the `user` table, location autocomplete using the Photon city search API

- **Validation:** Tested it on an account that hadn't received the onboarding yet
- **Decision:** Accepted / Modified
- **Reflection:** AI helped create a small but meaningful feature for when a user logs in for the first time or creates an account
- **Responsible Person:** Paul Fourment

Prompt 1

For the first sprint we need 2 features and one is the authentication. The other user stories are all a lot more work and I want a small feature I can add a user story for. I was thinking, once you register, you have a modal with an introduction or something? Do not start work yet.

Response 1

Suggested making the modal do one small job instead of only showing an introduction, so it counts as a real feature (it saves data, depends on the role, and can be tested and demoed):

First-login onboarding modal: right after registering, a modal welcomes the user and asks 2 quick questions for their role, which are saved.

- Job seeker: headline and location
- Recruiter: company name and job title
- "Save" or "Skip for now", and it only ever appears once.
- The answers become the start of the profile for US-03, so the work isn't thrown away.

Drafted user story:

US-16: First-login onboarding · Priority: Medium · Sprint: 1

As a new user, I want a quick welcome step after I register so that my account has basic details and I know what to do next.

Acceptance criteria

- [] After registering, a welcome modal opens automatically.
- [] Job seekers are asked for a headline and location; recruiters for their company and job title.
- [] The user can save the answers or skip.
- [] The modal never appears again after it has been saved or skipped, including on later logins.
- [] Saved answers are shown on the home page.

Tasks

- **16.1:** Add the onboarding columns to `user` (SQL for Supabase).
- **16.2:** Build the modal UI (native `<dialog>`, keyboard-accessible, glass style).
- **16.3:** Add the save and skip form actions.
- **16.4:** Show the saved answers on the home page and test that the modal only appears once.

Prompt 2

I've created a new branch `feature/login-onboarding`, start.

Response 2

Database: the answers are stored on the `user` table next to `role`, so Better Auth includes them with the logged-in user and no extra query is

needed.

```
-- US-11: onboarding answers + whether the user has been
alter table public."user"
  add column headline      text,
  add column location      text,
  add column company       text,
  add column job_title     text,
  add column onboarded_at timestamp;
```

Files

- `src/lib/components/OnboardingDialog.svelte`: the modal, built on the native `<dialog>` element (focus stays inside the modal, Esc closes it, dimmed backdrop) in the same glass style as the login pages, with a short entrance animation that is turned off for reduced motion.
- `src/lib/onboarding.ts`: the questions for each role, shared by the modal and the server so they stay in sync.
- `src/routes/+page.server.ts`: `onboard` and `skipOnboarding` actions. The server only accepts the fields for the user's role, requires them, and limits them to 100 characters. Both actions set `onboarded_at`, so the modal never shows again.
- `src/routes/+page.svelte`: opens the modal when `onboarded_at` is empty, otherwise shows the saved answers on the home card.
- `src/lib/server/auth.ts` and `src/lib/server/db/auth.schema.ts`: the new columns. They are marked read-only in Better Auth so they can't be set through the signup API.

Behaviour

- `Save` stores the answers; `Skip` stores nothing. Both mark the user as onboarded.
- Esc closes the modal for the current visit only; it comes back until the user saves or skips.

Test steps given

1. Log in with an account that hasn't been onboarded; the modal appears.
2. Save; the answers show on the home card. Log out and back in; no modal.
3. Register a recruiter and click Skip; the modal closes and doesn't return.

Prompt 3

Can we make the location field show options and autofill as you type, for all cities?

Response 3

Added city autocomplete using **Photon**, a free geocoding API built on OpenStreetMap (no API key, callable from the browser).

- `searchCities()` in `src/lib/onboarding.ts` asks Photon for up to 6 cities matching the text and formats them as "City, Province/State, Country".
- Results are biased toward Montréal, so local cities rank first without excluding the rest of the world (e.g. "montr" → Montreal, Montreal West, Montréal-Est).
- In the modal, the search runs once the user has typed 2 letters and paused for 250 ms. An older request is cancelled if the user keeps typing. Results appear in the input's native suggestion list (`<datalist>`), with "Remote" always offered.
- If Photon is unreachable, the field still works as a normal text box.
- The recruiter fields are tagged for browser autofill (`organization, organization-title`).

Testing: ran the search against the live API: "montr" → Montreal, Quebec, Canada; "tokyo" → Tokyo, Japan; "sao pa" → São Paulo,

Brazil; nonsense text → no suggestions.

Profile Management (US-04)

- **Purpose of AI Use:** Code generation for profile management page and the main page layout
- **AI-Suggested Content:** App layout with an account sidebar, routes for recruiters and job seekers, profile/experience/education tables (SQL to run on Supabase), profile view and section edit dialogs, photo upload to Supabase Storage, change email/password, delete account, recruiter view of a profile, page transitions, school/company/city/skill autocomplete with a custom dropdown
- **Validation:** Tested the profile management page, editing details and found it very clunky, added more prompts to make it a smoother experience and add all the sections needed for the future.
- **Decision:** Accepted / Modified
- **Reflection:** AI helped create the page and DB layout for the founding details for Job Seekers and Recruiters, with things like experience, education, skills, etc.
- **Responsible Person:** Paul Fourment

Prompt 1

I have created a new branch for feature/profile-management. Here is the user story to follow. I know the main page hasn't been sorted out yet, but if we can do a layout for it that would be great. It doesn't need content yet, just a simple layout: account card on the side, the jobs and stuff will be in the center.

(US-04 user story pasted: view and edit name, profile picture, email, password, phone number, location, skills, education and experience summary; changes persist; required fields show clear validation errors; users can edit only their own profile. Tasks 4.1–4.4.)

Response 1

Recruiters vs job seekers: use the same routes and let each page adapt to `user.role` (e.g. the home page shows "Jobs for you" or "Your job postings", the profile shows skills or company). Separate routes only make sense for pages one role must never see (e.g. a future recruiter-only "post a job" page), which get a one-line role check. Two parallel route trees would duplicate the layout, profile page and guards.

Layout (`src/routes/(app)/+layout.svelte`): every logged-in page shares a glass account card on the left (photo or initials, name, headline or job title, location, Home/Profile links, Log out) with the page content in the center. On phones the card moves to the top. Logout became its own `/logout` route so it works from any page.

Data (Task 4.1): a `profile` table with one row per user, linked to `user`. The onboarding answers moved from `user` into it, so `user` keeps only account data.

Profile page (Tasks 4.2–4.3): view mode with an Edit button; edit form for photo, name, phone, location (city autocomplete), headline/skills/education/experience for job seekers, company/job title for recruiters; separate forms to change email and password.

- Photo upload goes to a Supabase Storage bucket through its REST API from the server (secret key never reaches the browser), limited to PNG/JPG/WebP under 2 MB.
- Every action uses the logged-in user's id, never an id from the form, so users can only edit their own profile.

Tests (Task 4.4): validation moved into a pure function with unit tests using Node's built-in test runner (`pnpm test`), no new dependency.

Prompt 2

Can you add a simple page transition animation, quick and simple?

Response 2

Used the browser's View Transitions API through SvelteKit's `onNavigate` (the pattern from the SvelteKit docs, no library):

- The old page fades out (120 ms) and the new one fades in with a 6 px lift (200 ms).
- The account card is excluded from the transition so it stays still.
- Disabled for users with reduced motion enabled; browsers without support (Firefox) navigate normally.

Prompt 3

I want to avoid coming back to this page as much as possible, so I want to finalize it as much as possible right now. Remember the project description we were given: what other things would we need in the profile management area? For things like education, companies and skills, I'd like to make them simpler.

- Education: use possible APIs to autofill and show known schools
- Companies: same thing (like LinkedIn has) if an API is available
- Skills: how LinkedIn organizes skills

(Project description pasted)

Response 3

Data sources (tested live)

Field	Source	Result
Schools	Hipo university list (free, no key)	Good coverage incl. Québec CEGEPs; HTTP-only, so our server fetches it (<code>/api/schools</code>)
Companies	Clearbit autocomplete (free, no key)	Names + websites; logos shown from the company website's favicon
Skills	ESCO (EU skills taxonomy)	Rejected: "react" returned "react calmly in stressful situations"

Skills like LinkedIn: standard skill names picked from a search box, shown as a list, with up to 5 starred top skills shown first. Implemented with a built-in list of about 415 common skills (technical, tools, business, soft skills, languages), with custom skills still allowed.

Additions proposed for the profile (chosen: all except resume, which stays a placeholder for its own story):

1. Structured experience entries (title, company, employment type, location, start/end month or current, description)
2. Structured education entries (school, degree, field of study, years, description)
3. Job preferences (open to work, desired roles, job types, work mode, preferred locations), for job search filtering and matching later
4. Read-only recruiter view `/profile/[id]`, since the story says recruiters should understand the candidate's background
5. About section, links (LinkedIn, GitHub, portfolio), profile completeness bar, delete account

Implementation

- Profile organised like LinkedIn: each section is a card with its own Edit/Add button that opens a dialog. The open dialog is in the URL (e.g. `/profile?experience=new`), so back/Esc close it.
- Validation rewritten as one parser per section (dates in order and not in the future, years in range, links normalised, unknown choices dropped), covered by 6 unit test groups.
- Recruiter view: only recruiters can open it and only for job seekers; everyone else gets the same "not found" page so ids can't be probed.
- Delete account requires typing DELETE and the password; profile, experience and education rows are removed by the database's cascading deletes, and the photo is deleted from storage.

SQL to run in the Supabase SQL Editor

```

alter table public.profile
  drop column education,
  drop column experience,
  add column about          text,
  add column linkedin_url   text,
  add column github_url     text,
  add column website_url    text,
  add column company_domain text,
  add column top_skills     text[] not null default
  add column open_to_work   boolean not null default
  add column desired_roles  text[] not null default
  add column job_types      text[] not null default
  add column work_modes     text[] not null default
  add column preferred_locations text[] not null default

create table public.experience (
  id          uuid primary key default gen_random_uuid()
  user_id     text not null references public."user"()
  title       text not null,
  company     text not null,
  company_domain text,
  employment_type text,
  location    text,
  start_date  date not null,

```

```

end_date          date,
description        text,
created_at         timestamp not null default now(),
constraint experience_dates_check check (end_date is not null);
create index experience_user_id_idx on public.experience(user_id);
alter table public.experience enable row level security;

create table public.education (
  id                uuid primary key default gen_random_uuid(),
  user_id           text not null references public."user"(id),
  school            text not null,
  school_domain     text,
  degree            text,
  field_of_study    text,
  start_year        integer,
  end_year          integer,
  description        text,
  created_at        timestamp not null default now(),
  constraint education_years_check check (end_year is not null);
create index education_user_id_idx on public.education(user_id);
alter table public.education enable row level security;

```

Prompt 4

The preview of the suggestions is like the default browser dropdown and doesn't show the logos. Would it be possible to do it like LinkedIn, where it shows that information, so the options match the aesthetic?

Response 4

Replaced the browser's `<datalist>` with a custom `Combobox` component following the accessible combobox pattern (WAI-ARIA):

- Glass-styled list showing a logo, name and detail line: companies with their website, schools with province/country, cities with region/country; skills filter instantly with prefix matches first.
- Keyboard: arrow keys move, Enter picks, Esc closes the list without closing the dialog; in chip fields Enter on custom text still adds it.
- The list is rendered as a popover (top layer), so it appears above the dialog without being clipped, and flips above the field when there isn't room below.
- "Remote" is only offered when it matches what's typed.

Testing: checked in the browser: "shop" → Shopify / shopify.com with logo, keyboard selection stores the company's domain, "sherb" → Sherbrooke / Quebec, Canada, "py" → Python, pytest, PyTorch, and Esc behaviour inside the dialog.